

WuWei サーバー設定と保存領域

Table of Contents

1. 基本方針	1
2. 推奨ディレクトリ構成	1
3. Nginx / Apache での保護方針	2
4. load-file.cgi	2
5. アップロード処理に必要な外部コマンド	3
6. Resource definition	3
7. Note save	4
8. Export / Import	4
9. 動作確認	4
10. 注意事項	5

1. 基本方針

WuWei は、Web から直接参照される静的ファイル置場ではなく、ログイン済みユーザーごとの保存領域として `data/` を使用する。

`data/` 配下には、アップロードファイル、サムネイル、Office 変換 PDF、Resource definition、`note.txt` などが保存される。

原則として、クライアントは `data/` を直接読まず、`load-file.cgi`、`list-resource.cgi`、`load-note.cgi` などのサーバースクリプトを経由する。

2. 推奨ディレクトリ構成

```
wu_wei2/  
  app2/  
  cgi-bin/  
  server/  
  data/  
    {user_uuid}/  
      upload/  
        YYYY/MM/DD/{upload_uuid}/  
          manifest.json  
          {original_file}  
          thumbnail.jpg  
          preview.pdf  
      resource/  
        YYYY/MM/DD/{resource_uuid}/  
          resource.json  
      note/
```

```
YYYY/MM/DD/{note_uuid}/  
note.txt  
trash/
```

現在の通常保存では、ノート配下に `resource/{uploaded_uuid}/thumbnail.jpg` や `preview.pdf` を複製する方式ではない。

ファイル実体は `upload/` に置き、ホーム画面・検索用の Resource 定義は `resource/` に置く。

3. Nginx / Apache での保護方針

`data/` 配下を直接公開しない。直接アクセスを禁止し、必要なファイルだけを `load-file.cgi` で返す。

Nginx の場合、内部転送を使う構成では次の考え方になる。

```
location /wu_wei2/data/ {  
    deny all;  
}  
  
location /_wuwei2_data/ {  
    internal;  
    alias /var/www/wu_wei2/data/;  
}
```

`load-file.cgi` は認証とパス検査を行った上で、許可されたファイルを返す。

4. load-file.cgi

許可される `area` は次である。

- `upload`
- `note`
- `resource`
- `thumbnail`
- `content`

例を次に示す。

```
/wu_wei2/server/load-  
file.cgi?area=upload&path=2026/05/04/_resource_uuid/thumbnail.jpg&user_id={user_uuid}
```

`path` はユーザー領域内の相対パスである。...、絶対パス、制御文字を含むパスは拒否する。

5. アップロード処理に必要な外部コマンド

アップロード処理では、環境に応じて次のコマンドを使用する。

コマンド	用途
<code>python3</code>	CGI / Python スクリプト実行。
<code>file</code>	MIME type 判定。
<code>sha256sum</code>	ファイル同一性確認。
<code>pdftinfo</code>	PDF のページ数取得。
<code>pdftoppm</code>	PDF サムネイル生成。
<code>soffice</code> または <code>libreoffice</code>	Office 文書から <code>preview.pdf</code> への変換。
<code>zip, unzip</code>	ノートの export / import。

Amazon Linux 系では、利用できるパッケージ名がバージョンにより異なるため、`poppler-utils`、`libreoffice`、`ImageMagick` などの有無を確認する。

6. Resource definition

アップロード成功時には、次の Resource definition を作成する。

```
data/{user_uuid}/resource/YYYY/MM/DD/{resource_uuid}/resource.json
```

`resource.json` は、ホーム画面、検索、ノートへの挿入で使用する。

```
{
  "type": "Resource",
  "id": "_resource_uuid",
  "title": "document.pdf",
  "media": {
    "kind": "document",
    "pageCount": 18
  },
  "storage": {
    "managed": true,
    "copyPolicy": "reference",
    "manifest": {
      "area": "upload",
      "path": "YYYY/MM/DD/_resource_uuid/manifest.json"
    },
  },
  "files": [
    {
      "role": "original",
      "area": "upload",
    }
  ]
}
```

```
    "path": "YYYY/MM/DD/_resource_uuid/document.pdf"
  }
]
}
}
```

7. Note save

ノートは次に保存する。

```
data/{user_uuid}/note/YYYY/MM/DD/{note_uuid}/note.txt
```

note.txt には、メタデータ行と Base64 化されたノート JSON を保存する。

```
format_version 2
id _note_uuid
user_id {user_uuid}
name {note_name}
description {description}
json_encoding base64
json_base64 {base64-json}
```

8. Export / Import

可搬 ZIP は次を含む。

```
note.txt
upload/YYYY/MM/DD/{upload_uuid}/...
resource/YYYY/MM/DD/{resource_uuid}/resource.json
```

通常の **load-note.cgi** は ZIP を組み立てない。ZIP 化は **export-note.cgi** / **export-note.py** の役割である。

9. 動作確認

サーバースクリプトを確認する代表コマンドを次に示す。

```
curl -i 'https://www.example.jp/wu_wei2/server/list-resource.cgi'

curl -i 'https://www.example.jp/wu_wei2/server/load-
file.cgi?area=upload&path=2026/05/04/_resource_uuid/thumbnail.jpg'
```

ログイン Cookie が必要な構成では、ブラウザーのセッションまたは **curl** の Cookie オプションを指定し

て確認する。

10. 注意事項

- `resource.json` だけではファイル実体ではない。実体は `storage.files[]` が指す `upload/` 側にある。
- `upload/manifest.json` はアップロード bundle のメタデータであり、ホーム一覧の主入力ではない。
- `note.txt` の `json_base64` はノート構造の正本であり、Content ノードには compact resource が埋め込まれる。
- Contents axis の軸線は保存リンクではなく、`groups[]` から生成する pseudo link である。